

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Jayavarapu Raju	Reg. No.:	192325117
Section / Batch:	Slot-A	Date:	

Code of Conduct

I J.Raju(192325117) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J.Raju

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus: *"Data science is powerful, data science drives innovation, data science is evolving."*

The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences. The following statistics are obtained from the corpus:

- $C(\text{data}) = 3$
- $C(\text{data science}) = 3$
- $C(\text{science is}) = 2$
- $C(\text{science drives}) = 1$
- $C(\text{science drives innovation}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "science" are:

- $P(\text{is}) = 0.66$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability $P(\text{science} | \text{data})$. Interpret how this probability influences next-word prediction in a mobile keyboard application.
 2. A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for real-time predictive text systems.
 3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.
 4. Calculate the Entropy of the distribution $P(\text{is})=0.66$ and $P(\text{drives})=0.33$. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.
-

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses.

Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.

Given:

- $P(\text{book} \mid \text{VB}) = 0.6$
- $P(\text{book} \mid \text{NN}) = 0.4$
- $P(\text{Start} \rightarrow \text{VB}) = 0.5$
- $P(\text{Start} \rightarrow \text{NN}) = 0.5$

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.
 2. Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.
 3. Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.
 4. Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.
-

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.
2. Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.
3. Assuming the corpus contains the words:
 - economic (120 occurrences)
 - growth (450 occurrences)
 - increases (210 occurrences)
 - employment (380 occurrences)

Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

4. Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.
-

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: Smart Mobile Keyboard Prediction System

Corpus:

“Data science is powerful, data science drives innovation, data science is evolving.”

Given:

- $C(\text{data})=3$
- $C(\text{data science})=3$
- $C(\text{science is})=2$
- $C(\text{science drives})=1$
- $C(\text{science drives innovation})=1$

Interpolation weights:

- $\lambda_1=0.5$ Trigram
 - $\lambda_2=0.3$ Bigram
 - $\lambda_3=0.2$ Unigram
-

1. MLE of Bigram Probability $P(\text{science}|\text{data})$

The MLE formula for a bigram is:

$$P(w_i|w_{i-1})=C(w_{i-1})C(w_{i-1},w_i)$$

Therefore:

$$P(\text{science}|\text{data})=C(\text{data})C(\text{data science})$$

Substituting the given values:

$$P(\text{science}|\text{data})=\frac{3}{3} P(\text{science}|\text{data})=1.0$$

Interpretation

The probability is **1.0 or 100%** because every occurrence of **data** in the training corpus is followed by **science**.

Therefore, when the keyboard sees:

data

it can confidently predict:

science

This increases prediction accuracy and makes the keyboard respond quickly with a highly probable next word.

2. Backoff for "data science improves"

The user enters:

data science improves

The trigram:

data science improves

does **not occur** in the training corpus.

Therefore:

$$P(\text{improves}|\text{data},\text{science})=0$$

An unsmoothed trigram model cannot make a useful prediction.

Backoff process

The model first tries the trigram:

$$P(\text{improves}|\text{data},\text{science})$$

It is unseen, so:

$$=0$$

The model backs off to the bigram:

$$P(\text{improves}|\text{science})$$

There is no **science improves** bigram in the corpus, so:

$$=0$$

Finally, the model backs off to the unigram:

$$P(\text{improves})$$

The word **improves** does not occur in the training corpus.

Therefore:

$$P(\text{improves})=0$$

Hence the estimated probability is:

$$P(\text{improves}|\text{data},\text{science})=0$$

Why is backoff essential?

Backoff is important because real users constantly type **new and unseen combinations**.

For example:

data science improves

may not exist in the training corpus even though it is grammatically meaningful.

A practical keyboard therefore uses:

Trigram → Bigram → Unigram

This allows the system to make a prediction even when the exact word sequence was not seen during training.

Note: In this particular tiny corpus, *improves* itself is unseen, so even the unigram fallback is zero. A real keyboard would use a much larger vocabulary and usually smoothing as well.

3. Deleted Interpolation for "data science is"

We want:

$P(\text{is}|\text{data}, \text{science})$

The interpolation formula is:

$P = \lambda_1 P_{\text{tri}} + \lambda_2 P_{\text{bi}} + \lambda_3 P_{\text{uni}}$

where:

- $\lambda_1 = 0.5$
- $\lambda_2 = 0.3$
- $\lambda_3 = 0.2$

Trigram probability

From the corpus:

data science is

occurs twice.

The bigram:

data science

occurs three times.

Therefore:

$P_{\text{tri}}(\text{is}|\text{data}, \text{science}) = \frac{C(\text{data science is})}{C(\text{data science})} = \frac{2}{3} \Rightarrow P_{\text{tri}} = 0.667$

Bigram probability

$P_{\text{bi}}(\text{is}|\text{science}) = \frac{C(\text{science is})}{C(\text{science})}$

science occurs 3 times.

Therefore:

$$P_{bi}=32=0.667$$

Unigram probability

The corpus contains 13 words in total.

is occurs twice.

Therefore:

$$P_{uni}(is)=\frac{2}{13} \quad P_{uni}=0.154$$

Interpolation

$$P(is|data,science)=(0.5)(0.667)+(0.3)(0.667)+(0.2)(0.154) = 0.3335+0.2001+0.0308 \quad P(is|data,science) \approx 0.564$$

Interpretation

Deleted interpolation combines:

- **Trigram:** detailed contextual information.
- **Bigram:** broader contextual information.
- **Unigram:** general word frequency.

Therefore, if the trigram is unreliable or sparse, the lower-order models still contribute probability.

This provides a balance between:

Reliability + Coverage

and is especially useful for sparse datasets.

4. Entropy of $P(is)=0.66$ and $P(drives)=0.33$

Entropy is:

$$H = -\sum P(x) \log_2 P(x)$$

Therefore:

$$H = -(0.66 \log_2(0.66) + 0.33 \log_2(0.33))$$

Calculate each part:

$$0.66 \log_2(0.66) \approx -0.399 \quad 0.33 \log_2(0.33) \approx -0.528$$

Therefore:

$$H \approx 0.399 + 0.528 \quad H \approx 0.927 \text{ bits}$$

Because the probabilities given sum to **0.99 rather than exactly 1.0**, a normalized calculation gives approximately:

$$H \approx 0.915 \text{ bits}$$

Interpretation

The maximum entropy for two equally likely outcomes is:

$H_{\max}=1$ bit

Our entropy is close to 1 bit, meaning there is **moderate uncertainty** between **is** and **drives**.

However, **is** has a higher probability:

$P(\text{is})=0.66$

so the keyboard has more confidence in predicting **is**.

Role of entropy

Entropy helps the keyboard determine how certain its predictions are.

- **Low entropy** → **high confidence**
- **High entropy** → **high uncertainty**

Therefore, entropy can help an intelligent keyboard decide whether to show one strong prediction or several alternative suggestions.

CASE STUDY 2: AI-Powered Customer Support Chatbot

Sentences:

1. **Book a flight ticket now.**
2. **This book is interesting.**

Given:

$P(\text{book}|\text{VB})=0.6$ $P(\text{book}|\text{NN})=0.4$ $P(\text{Start} \rightarrow \text{VB})=0.5$ $P(\text{Start} \rightarrow \text{NN})=0.5$

1. Penn Treebank POS Tags

Sentence 1

Book a flight ticket now.

Word	POS Tag	Meaning
Book	VB	Base-form verb
a	DT	Determiner
flight	NN	Noun
ticket	NN	Noun
now	RB	Adverb

Therefore:

Book/VB a/DT flight/NN ticket/NN now/RB

Why is "book" a verb?

The sentence is an **imperative command**.

"Book a flight ticket."

Here, **Book** means **make a reservation**, so it functions as a verb.

Sentence 2

This book is interesting.

Word	POS Tag	Meaning
This	DT	Determiner
book	NN	Noun
is	VBZ	Verb
interesting	JJ	Adjective

Therefore:

This/DT book/NN is/VBZ interesting/JJ

Why is "book" a noun?

Here, **book** refers to a physical/written object.

The structure is:

This **book** is interesting.

book is the subject/noun of the sentence.

2. HMM Probability of "book" as Verb

The HMM combines:

1. Initial/transition probability.
2. Emission probability.

For **book** as a verb:

$P(\text{VB}|\text{Start}) \times P(\text{book}|\text{VB})$

Given:

$P(\text{Start} \rightarrow \text{VB}) = 0.5$

and:

$P(\text{book}|\text{VB}) = 0.6$

Therefore:

$$P(\text{VB}, \text{book}) = 0.5 \times 0.6 \quad P(\text{VB}, \text{book}) = 0.30$$

For comparison, noun:

$$P(\text{NN}, \text{book}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} | \text{NN}) = 0.5 \times 0.4 \quad P(\text{NN}, \text{book}) = 0.20$$

Therefore:

$$0.30 > 0.20$$

So the HMM favors:

VB

Interpretation

The model considers **book** more likely to be a verb because:

$$P(\text{book} | \text{VB}) = 0.6$$

is greater than:

$$P(\text{book} | \text{NN}) = 0.4$$

Therefore, for the sentence beginning with **Book**, the HMM selects **VB**.

3. Rule-Based vs Stochastic POS Tagging

Feature	Rule-Based	Stochastic/HMM
Method	Hand-written rules	Probabilities
Training data	Not necessarily required	Required
Ambiguity	Limited	Better handling
Context	Grammar rules	Statistical context
Scalability	Difficult	Better
Maintenance	Rules must be updated	Retrain/update model

Rule-based approach

A rule might say:

If a word occurs at the beginning of an imperative sentence, it can be a verb.

This can correctly identify:

Book a flight.

But hand-written rules become difficult when the system has thousands of linguistic patterns.

HMM approach

The HMM uses probabilities and context to determine the most likely tag.

For a large-scale chatbot, I would recommend:

Stochastic/statistical or modern probabilistic tagging

because it can learn patterns from large amounts of data and handle lexical ambiguity more effectively.

4. Importance of Standardized POS Tagsets

The **Penn Treebank tagset** provides consistent grammatical labels such as:

- **NN** → noun
- **VB** → verb
- **JJ** → adjective
- **RB** → adverb
- **PRP** → pronoun
- **DT** → determiner

This consistency is important for a chatbot.

Intent detection

POS information helps distinguish:

Book a flight

from:

This book is interesting

The word **book** has different grammatical roles.

Response generation

The chatbot can understand the structure of a sentence and generate grammatically appropriate responses.

Information extraction

POS tags help identify:

- Actions → verbs
- Objects → nouns

- Descriptions → adjectives
- Time information → adverbs

Overall benefit

Standardized POS tags provide a common representation that improves:

Intent Detection + Language Understanding + Response Generation

CASE STUDY 3: News Analytics and POS Tag Correction

Sentence:

Economic growth increases employment.

Initial tags:

economic/JJ
growth/NN
increases/NNS
employment/NN

Transformation rule:

Change NNS to VBZ if the preceding word is tagged as NN.

1. Apply the Transformation Rule

Initial sequence:

JJNNNNSNN

The word **increases** is currently:

increases/NNS

Its preceding word is:

growth/NN

The transformation rule says:

Change NNS → VBZ if previous word is NN.

Therefore:

increases/NNS

becomes:

increases/VBZ

Final sequence

economic/JJ growth/NN increases/VBZ employment/NN

Why is this linguistically appropriate?

growth is the singular subject.

The verb must agree with the singular subject:

Economic growth **increases** employment.

Therefore **increases** is a **third-person singular present-tense verb**, represented by:

VBZ

2. Analyze the Initial Errors

The initial tagging was:

economic/JJ

growth/NN

increases/NNS

employment/NN

The main error is:

increases/NNS

NNS means **plural noun**.

But in the sentence:

Economic growth increases employment.

increases describes the action performed by the subject **growth**.

Therefore:

increases → VBZ

Grammatical structure

Economic → JJ

growth → NN

increases → VBZ

employment → NN

Meaning:

- **economic** describes **growth**.
- **growth** is the subject.
- **increases** is the verb.
- **employment** is the object.

Thus the corrected sequence is grammatically appropriate.

3. Word Frequency Distribution

Given:

Word	Frequency
economic	120
growth	450
increases	210
employment	380

Total frequency:

$$120+450+210+380=1160$$

The frequency distribution is:

$$P(\text{word}) = \frac{\text{Total frequency}}{C(\text{word})}$$

Economic

$$P(\text{economic}) = \frac{120}{1160} \quad P(\text{economic}) \approx 0.1034$$

Growth

$$P(\text{growth}) = \frac{450}{1160} \quad P(\text{growth}) \approx 0.3879$$

Increases

$$P(\text{increases}) = \frac{210}{1160} \quad P(\text{increases}) \approx 0.1810$$

Employment

$$P(\text{employment}) = \frac{380}{1160} \quad P(\text{employment}) \approx 0.3276$$

Distribution

Word	Count	Probability	Percentage
economic	120	0.1034	10.34%
growth	450	0.3879	38.79%
increases	210	0.1810	18.10%
employment	380	0.3276	32.76%
Total	1160	1.0000	100%

How frequency helps POS tagging

Frequency information helps a statistical POS tagger estimate which tags are likely for a word.

For example, if **increases** occurs frequently as **VBZ** in the training corpus, then the model can assign a higher probability to:

$P(\text{VBZ}|\text{increases})$

However, frequency alone is not enough. **Context and transition probabilities** are also important.

For example:

growth/NN increases/VBZ

provides strong grammatical evidence that **increases** is a verb.

4. Transformation-Based Tagging and Entropy

Transformation-Based Tagging improves an initial tagging result by applying learned correction rules.

Before transformation

economic/JJ
growth/NN
increases/NNS
employment/NN

The problematic tag is:

increases/NNS

After transformation

economic/JJ
growth/NN
increases/VBZ
employment/NN

Thus the transformation corrects the grammatical error.

Why it improves accuracy

The initial tagger may make mistakes because a word can have multiple possible grammatical categories.

Transformation-based tagging uses **context-sensitive rules** to correct these mistakes.

The process is:

Initial POS tagging



Find incorrect patterns



Apply learned transformation



Correct POS sequence

This is particularly useful for large collections of news articles where a small improvement in tagging accuracy can significantly improve downstream NLP tasks.

Entropy Before and After Transformation

Entropy measures uncertainty.

The general formula is:

$$H = -\sum_i P(t_i) \log_2 P(t_i)$$

where t_i represents a possible POS tag.

Suppose before correction, the system is uncertain between **NNS** and **VBZ**:

$$P(\text{NNS}) = 0.5 \quad P(\text{VBZ}) = 0.5$$

Then:

$$H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) \quad H = 1 \text{ bit}$$

This represents maximum uncertainty for two equally likely tags.

After applying the transformation rule, suppose the model becomes highly confident:

$$P(\text{VBZ}) = 0.95 \quad P(\text{NNS}) = 0.05$$

Then:

$$H = -(0.95 \log_2 0.95 + 0.05 \log_2 0.05) \quad H \approx 0.286 \text{ bits}$$

So:

1.0 bit $\rightarrow$ 0.286 bit

Interpretation

The decrease in entropy indicates that the system has become **more confident about the correct POS tag**.

Therefore:

Transformation $\rightarrow$ lower uncertainty $\rightarrow$ better tagging confidence